Лабораторна робота №4

Тема: Модульне програмування, низькорівневі порозрядні операції, бітові маски та командний процес розробки через Pull Request у Git

Посилання на GitHub

Мета роботи

Опанувати принципи функціональної декомпозиції програм: проектування підпрограм із розділенням на прототипи та реалізацію.
Дослідити роботу автоматичних локальних змінних на стеку та використання специфікатора static для збереження стану функції між викликами.
Опанувати порозрядні операції &, |, ^, ~, <<, >> та чотири канонічні операції над регістрами: встановлення, скидання, інвертування і перевірку бітів за допомогою масок.
Опанувати командний процес інтеграції коду через Pull Request на GitHub.
Підготувати підсумкову документацію у форматі PDF.

Етап 1. Функціональна декомпозиція та збереження стану (task1.c)
Функція обчислення гіпотенузи за двома катетами. Статична змінна накопичує середнє значення всіх розрахованих гіпотенуз.

Код програми task1.c

```c
#include <stdio.h>
#include <math.h>

// Прототипи функцій
double calc_hypotenuse(double leg_a, double leg_b);
double update_average(double value);

int main(void) {
    int count;

    printf("How many triangles to process? ");
    if (scanf("%d", &count) != 1 || count <= 0) {
        printf("Error: count must be a positive integer.\n");
        return 1;
    }

    for (int i = 1; i <= count; i++) {
        double leg_a, leg_b;

        printf("Triangle %d, enter legs a and b: ", i);
        if (scanf("%lf %lf", &leg_a, &leg_b) != 2) {
            printf("Error: legs must be numbers.\n");
            return 1;
        }
        if (leg_a <= 0 || leg_b <= 0) {
            printf("Error: legs must be greater than zero.\n");
            return 1;
        }

        double hypotenuse = calc_hypotenuse(leg_a, leg_b);
        double average = update_average(hypotenuse);

        printf("Hypotenuse = %.3f, running average = %.3f\n",
               hypotenuse, average);
    }

    return 0;
}
```

```c

// Розрахунок гіпотенузи, аргументи передаються за значенням
double calc_hypotenuse(double leg_a, double leg_b) {
    return sqrt(leg_a * leg_a + leg_b * leg_b);
}

// Статичні змінні зберігають стан між викликами
double update_average(double value) {
    static double sum = 0.0;
    static int calls = 0;

    sum += value;
    calls++;

    return sum / calls;
}
```

Компіляція та запуск: gcc -Wall -Wextra task1.c -o task1 -lm
.\task1.exe

Результат виконання

```
PS C:\Users\illia\projects\programming-sem1\lab04> .\task1.exe
How many triangles to process? 3
Triangle 1, enter legs a and b: 3 4
Hypotenuse = 5.000, running average = 5.000
Triangle 2, enter legs a and b: 5 12
Hypotenuse = 13.000, running average = 9.000
Triangle 3, enter legs a and b: 8 15
Hypotenuse = 17.000, running average = 11.667
```

Пояснення програми

Програма розбита на підпрограми. Прототипи функцій calc_hypotenuse() та update_average() розміщено на початку файлу, до функції main. Це необхідно, щоб компілятор знав типи аргументів і значення, що повертається, ще до першого виклику функції. Функція main містить лише інтерфейсну логіку: введення кількості трикутників і катетів, перевірку введених даних, виклик обчислювальних функцій і друк результатів.

Функція calc_hypotenuse() обчислює гіпотенузу за формулою $c = \sqrt{(a^2 + b^2)}$. Аргументи передаються за значенням (Pass-by-Value): функція отримує копії катетів, тому зміни всередині функції не впливають на змінні в main.

Функція update_average() містить статичні локальні змінні sum та calls зі специфікатором static. На відміну від звичайних автоматичних змінних, які створюються на стеку при кожному виклику й зникають після виходу

з функції, статичні змінні зберігаються в сегменті даних і живуть увесь час роботи програми. Тому сума та кількість викликів накопичуються між викликами без використання глобальних змінних. Середнє значення обчислюється як sum / calls.
Захист вхідних даних: перевіряється значення, яке повертає scanf, а також умова, що кількість трикутників і катети більші за нуль.
Результат: для катетів (3; 4), (5; 12) та (8; 15) гіпотенузи дорівнюють 5, 13 і 17, а середнє значення послідовно змінюється 5.000, 9.000, 11.667.

Етап 2. Емуляція 32-бітного апаратного регістра GPIO (task2.c), варіант 5 Встановити біт 7, скинути біт 2, інвертувати біт 15, перевірити стан біта 20.

Код програми task2.c

```c
#include <stdio.h>
#include <stdint.h>

// Прототипи функцій
uint32_t set_bit(uint32_t reg, int n);
uint32_t clear_bit(uint32_t reg, int n);
uint32_t toggle_bit(uint32_t reg, int n);
int check_bit(uint32_t reg, int n);
void print_binary(uint32_t reg);

int main(void) {
    const int SET_POS = 7;
    const int CLEAR_POS = 2;
    const int TOGGLE_POS = 15;
    const int CHECK_POS = 20;

    // Початковий стан: встановлені біти 2 та 20
    uint32_t reg = 0x00100004;

    printf("Initial   : ");
    print_binary(reg);
    printf("  (0x%08X)\n", (unsigned int)reg);

    reg = set_bit(reg, SET_POS);
    printf("Set %2d    : ", SET_POS);
    print_binary(reg);
    printf("  (0x%08X)\n", (unsigned int)reg);

    reg = clear_bit(reg, CLEAR_POS);
    printf("Clear %2d  : ", CLEAR_POS);
    print_binary(reg);
    printf("  (0x%08X)\n", (unsigned int)reg);

    reg = toggle_bit(reg, TOGGLE_POS);
    printf("Toggle %2d : ", TOGGLE_POS);
    print_binary(reg);
    printf("  (0x%08X)\n", (unsigned int)reg);

    printf("Check bit %d = %d\n", CHECK_POS, check_bit(reg, CHECK_POS));
```

```c

    return 0;
}

// Встановлення біта: побітове АБО з маскою
uint32_t set_bit(uint32_t reg, int n) {
    return reg | ((uint32_t)1 << n);
}

// Скидання біта: побітове І з інверсією маски
uint32_t clear_bit(uint32_t reg, int n) {
    return reg & ~((uint32_t)1 << n);
}

// Інвертування біта: побітове XOR з маскою
uint32_t toggle_bit(uint32_t reg, int n) {
    return reg ^ ((uint32_t)1 << n);
}

// Перевірка біта: повертає 1 або 0
int check_bit(uint32_t reg, int n) {
    return (reg & ((uint32_t)1 << n)) != 0;
}

// Вивід усіх 32 бітів, групами по 8
void print_binary(uint32_t reg) {
    for (int i = 31; i >= 0; i--) {
        putchar(((reg >> i) & 1) ? '1' : '0');
        if (i % 8 == 0 && i != 0) {
            putchar(' ');
        }
    }
}
```

Компіляція та запуск: gcc -Wall -Wextra task2.c -o task2
.\task2.exe

Результат виконання

```
PS C:\Users\illia\projects\programming-sem1\lab04> .\task2.exe
Initial   : 00000000 00010000 00000000 00000100  (0x00100004)
Set  7    : 00000000 00010000 00000000 10000100  (0x00100084)
Clear  2  : 00000000 00010000 00000000 10000000  (0x00100080)
Toggle 15 : 00000000 00010000 10000000 10000000  (0x00108080)
Check bit 20 = 1
```

Пояснення програми

Регістр емулюється змінною типу uint32_t із бібліотеки stdint.h, що гарантує рівно 32 біти без знака. Програма складається з п'яти незалежних підпрограм, прототипи яких розміщено перед main. Функція set_bit() встановлює біт N в одиницю за допомогою побітового «АБО» з маскою (1 << N). Функція clear_bit() скидає біт у нуль за допомогою побітового «І» з інверсією маски ~(1 << N). Функція toggle_bit() інвертує біт за допомогою побітового «XOR» з маскою (1 << N). Функція check_bit() повертає 1 або 0, виконуючи побітове «І» з маскою і порівнюючи результат з нулем. Функція print_binary() виводить усі 32 біти регістра групами по 8.

Розрахунок масок для варіанта 5:

- встановлення біта 7: маска 1 << 7 = 0x00000080;

- скидання біта 2: маска 1 << 2 = 0x00000004, її інверсія ~(1 << 2) = 0xFFFFFFFB;

- інвертування біта 15: маска 1 << 15 = 0x00008000;

- перевірка біта 20: маска 1 << 20 = 0x00100000.

Початкове значення регістра 0x00100004 (встановлені біти 2 та 20). Після set_bit(7) регістр дорівнює 0x00100084, після clear_bit(2) дорівнює 0x00100080, після toggle_bit(15) дорівнює 0x00108080. check_bit(20) повертає 1, бо біт 20 лишився встановленим.

У виведеному двійковому коді видно, що на кожному кроці змінюється лише цільовий біт, а сусідні розряди не порушуються.

Етап 3. Пакування та розпакування телеметрії в бітове слово (task3.c), варіант 5 Структура 16-бітного слова: біти 0..11 напруга батареї в мілівольтах (0..4095); біти 12..14 стан заряду (0..7); біт 15 прапорець заряджання (0..1).

Код програми task3.c:

```c
#include <stdio.h>
#include <stdint.h>

#define VOLTAGE_BITS   12
#define STATE_SHIFT    12
#define CHARGING_SHIFT 15

// Прототипи функцій
uint16_t pack_telemetry(unsigned int voltage, unsigned int state,
                        unsigned int charging);
void unpack_telemetry(uint16_t packet, unsigned int *voltage,
                      unsigned int *state, unsigned int *charging);

int main(void) {
    const unsigned int MAX_VOLTAGE = 4095;
    const unsigned int MAX_STATE = 7;
    const unsigned int MAX_CHARGING = 1;

    unsigned int voltage, state, charging;

    printf("Enter battery voltage in mV (0..%u): ", MAX_VOLTAGE);
    if (scanf("%u", &voltage) != 1 || voltage > MAX_VOLTAGE) {
        printf("Error: voltage must be in range 0..%u.\n", MAX_VOLTAGE);
        return 1;
    }

    printf("Enter charge state (0..%u): ", MAX_STATE);
    if (scanf("%u", &state) != 1 || state > MAX_STATE) {
        printf("Error: state must be in range 0..%u.\n", MAX_STATE);
        return 1;
    }

    printf("Enter charging flag (0..%u): ", MAX_CHARGING);
    if (scanf("%u", &charging) != 1 || charging > MAX_CHARGING) {
        printf("Error: flag must be 0 or 1.\n");
        return 1;
    }

```

```c
    uint16_t packet = pack_telemetry(voltage, state, charging);

    unsigned int new_voltage, new_state, new_charging;
    unpack_telemetry(packet, &new_voltage, &new_state, &new_charging);

    printf("\nOriginal : voltage = %u, state = %u, charging = %u\n",
           voltage, state, charging);
    printf("Packet   : HEX: 0x%04X\n", (unsigned int)packet);
    printf("Unpacked : voltage = %u, state = %u, charging = %u\n",
           new_voltage, new_state, new_charging);

    if (voltage == new_voltage && state == new_state
        && charging == new_charging) {
        printf("Check: values are identical.\n");
    } else {
        printf("Check: values differ!\n");
    }

    return 0;
}

// Пакування: зсув ліворуч і побітове АБО
uint16_t pack_telemetry(unsigned int voltage, unsigned int state,
                        unsigned int charging) {
    return (uint16_t)(voltage
                    | (state << STATE_SHIFT)
                    | (charging << CHARGING_SHIFT));
}

// Розпакування: маски та зсув праворуч
void unpack_telemetry(uint16_t packet, unsigned int *voltage,
                      unsigned int *state, unsigned int *charging) {
    *voltage = packet & 0x0FFF;
    *state = (packet >> STATE_SHIFT) & 0x07;
    *charging = (packet >> CHARGING_SHIFT) & 0x01;
}
```

Компіляція та запуск: gcc -Wall -Wextra task3.c -o task3
.\task3.exe

Результат виконання

```
PS C:\Users\illia\projects\programming-sem1\lab04> .\task3.exe
Enter battery voltage in mV (0..4095): 3700
Enter charge state (0..7): 5
Enter charging flag (0..1): 1

Original : voltage = 3700, state = 5, charging = 1
Packet   : HEX: 0xDE74
Unpacked : voltage = 3700, state = 5, charging = 1
Check: values are identical.
```

Пояснення програми

Три параметри пакуються в одне беззнакове 16-бітне слово типу uint16_t. Поле напруги займає біти 0..11, поле стану заряду біти 12..14, прапорець заряджання біт 15.

Функція pack_telemetry() розташовує кожне поле у відведеному діапазоні бітів: напруга залишається в молодших розрядах, стан заряду зсувається ліворуч на 12 позицій (state << 12), прапорець заряджання зсувається на 15 позицій (charging << 15). Поля об'єднуються побітовим «АБО» (|).

Функція unpack_telemetry() виконує зворотну операцію за допомогою масок і зсуву праворуч: напруга виділяється маскою 0x0FFF, стан заряду (packet >> 12) & 0x07, прапорець заряджання (packet >> 15) & 0x01. Відновлені значення повертаються через вказівники.

Захист вхідних даних: перевіряється значення, що повертає scanf, і належність кожного значення своєму діапазону (напруга 0..4095, стан 0..7, прапорець 0 або 1). Без цього надто велике значення «залізло» б у сусіднє поле слова.

Розрахунок для напруги 3700 мВ, стану 5 та прапорця 1: 3700 = 0x0E74, стан 5 у бітах 12..14 дає 0x5000, прапорець 1 у біті 15 дає 0x8000. Пакет дорівнює 0x0E74 | 0x5000 | 0x8000 = 0xDE74. Після розпакування отримано ті самі значення (3700, 5, 1), що підтверджує їхню повну ідентичність.

Висновок

Під час виконання лабораторної роботи я опанував модульне програмування та порозрядні операції мови С.

На етапі 1 розбив програму на підпрограми з прототипами, передавав аргументи за значенням і використав статичні локальні змінні для накопичення середнього значення гіпотенуз між викликами без глобальних змінних.

На етапі 2 реалізував чотири канонічні операції над 32-бітним регістром: встановлення, скидання, інвертування та перевірку біта за допомогою масок, і переконався, що зміна цільового біта не порушує сусідні розряди.

На етапі 3 запакував напругу батареї, стан заряду й прапорець заряджання в одне 16-бітне слово за допомогою зсувів і побітового «АБО», а потім розпакував їх масками та зсувом праворуч, підтвердивши ідентичність значень.

Також опанував командний процес розробки: створив гілку lab04, зафіксував роботу атомарними комітами та оформив Pull Request на GitHub.